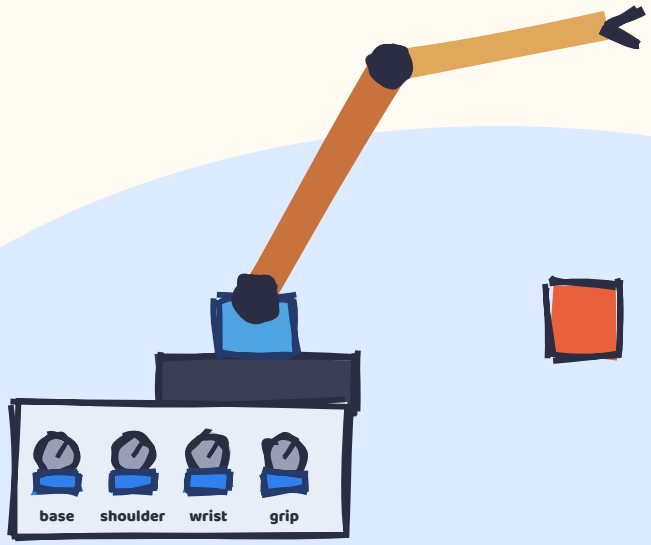
Tech Leaders · Class 3

Pick-and-Place Robotic Arm with Record & Replay

Teach it once, it repeats forever

4 servos, 4 DOF | EEPROM memory | Arm trigonometry



Did you know?
Factory arms are 'taught' by an operator with a hand-held teach pendant, then repeat the motion thousands of times.

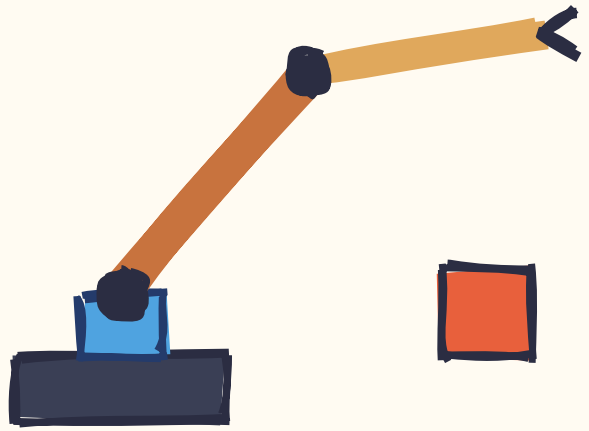
LEARN

Engineering challenge

Same task, 1000 times a day

The factory problem

A packing line must move every box from a conveyor to a tray. A person gets tired and makes mistakes. A robot never gets bored, but someone has to **teach** it the movement first, without writing new code every time.



Hire more people?

Code every angle?

Show it once, record, replay!

Today: you build a 4-joint arm with a matching knob controller, **record** a sequence of poses in memory and **replay** it to move objects.

LEARN

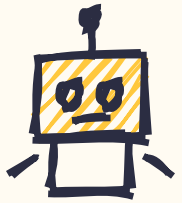
Learning aims

Today I will learn to...

1. Explain **degrees of freedom**, torque and why servos are sized differently
2. Control 4 servos live from 4 **potentiometers** using map()
3. Store poses in **EEPROM** so they survive a power cut
4. Replay smoothly with **interpolation**
5. Calculate reach with **trigonometry** and measure accuracy

Keywords

DOF · servo · torque · potentiometer · EEPROM · interpolation · repeatability



Watch and think

Video time!



Pick and Place Robot Arms: How Do Robotic Arms Work?

Prolific 3D Tech · about 2 minutes · youtu.be/AgTvrmD1bDI

Watch for...

1. How many **joints** does the arm use?
2. What is the **end effector**?
3. Why do factories use pick-and-place robots?

Scan to watch

After the video: Our arm records and replays moves. List the joint moves needed to pick up a pen and drop it in a cup.

Real world

Arms that record and replay



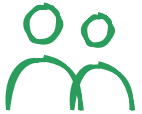
Car factories
Welding and painting robots repeat taught paths



Warehouses
Pick-and-place arms pack online orders



Surgery
Surgeons' hand motions are scaled down by robot arms



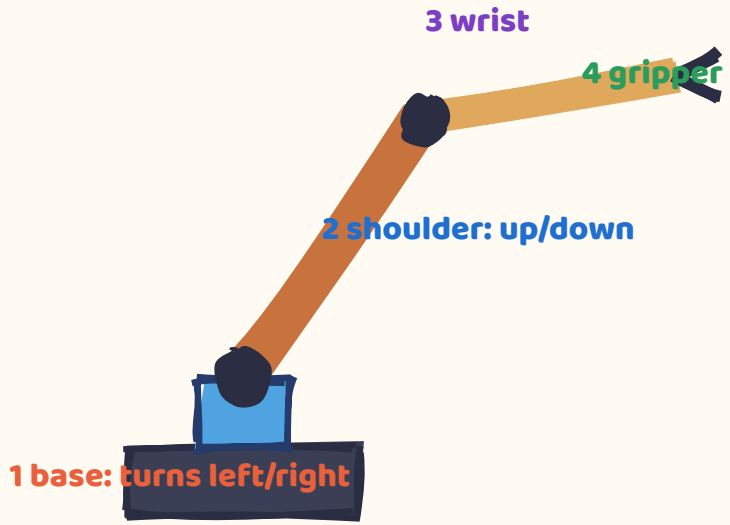
Cobots
Small 'collaborative' arms are taught by moving them by hand

? Which jobs in Pakistan's factories (textiles, food packing) could a pick-and-place arm help with? Who might be affected?

Think deeper
Automation changes jobs. Engineers should design with workers, not against them.

Mechanics

Degrees of freedom: one joint, one movement



Joint	Servo	Signal	Knob
Base	MG995 (metal gear)	pin 3	A0
Shoulder	MG995 (metal gear)	pin 5	A1
Wrist	SG90	pin 6	A2
Gripper	SG90	pin 9	A3

4 DOF

Our arm has 4 degrees of freedom. A human arm has about 7, which is why you can reach around obstacles. Industrial arms usually have 6.

Physics

Torque: why the shoulder works hardest



torque = force × distance

Holding the same block twice as far away needs twice the torque.

Servo sizes

SG90: about **1.8 kg·cm**. MG995: about **10 kg·cm** (datasheet values). The shoulder carries the whole forearm, so it gets the strong servo.

Design rules

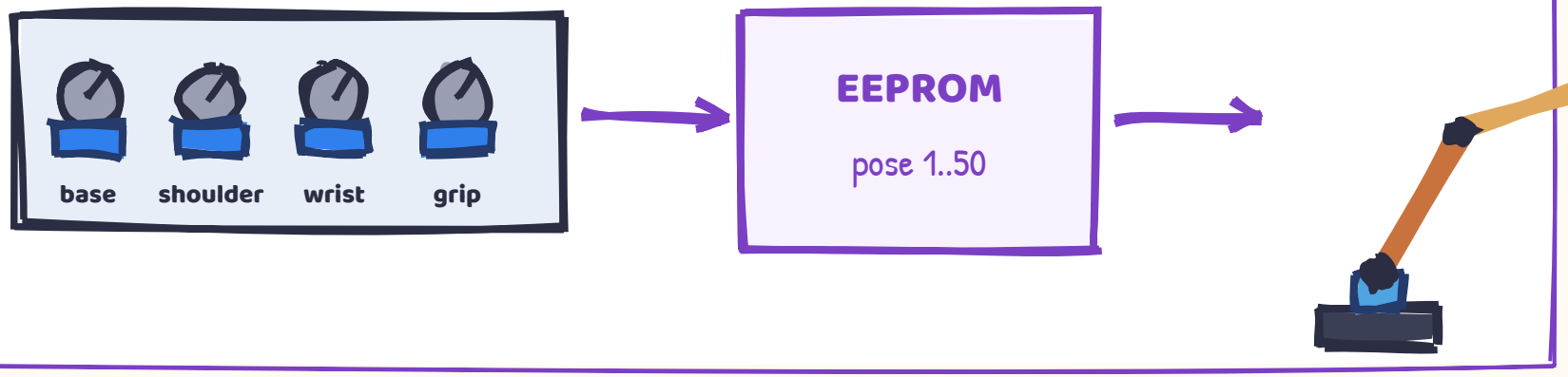
Keep links **short and light** (card, foam board, or 3D-printed with low infill). Put heavy parts close to the base.

Try it

Hold a water bottle with your arm straight, then bent. Which tires faster?

Big idea

Teach mode and play mode



Puppet controller
Arrange the 4 knobs like the arm's joints. Teaching becomes natural, like a puppet.

Teach (record)

Move the knobs: the arm copies you live. Press **REC** to save the current pose. Repeat for each step of the task.

Play (replay)

Press **PLAY**: the arm moves smoothly through every saved pose, in order, and can loop forever.

Hold both buttons
for 2 seconds to erase the recording and start again.

Computing

Memory that survives a power cut

EEPROM address	Stores
0	number of saved poses (0-50)
1, 2, 3, 4	pose 1: base, shoulder, wrist, grip
5, 6, 7, 8	pose 2
…	…
197-200	pose 50

Why EEPROM?

Normal variables (RAM) are wiped when power goes off. The Uno's **1024-byte EEPROM** keeps data for years.

One byte per angle

An angle is 0-180, which fits in one byte (0-255). 4 joints × 50 poses = 200 bytes.

Wear out!

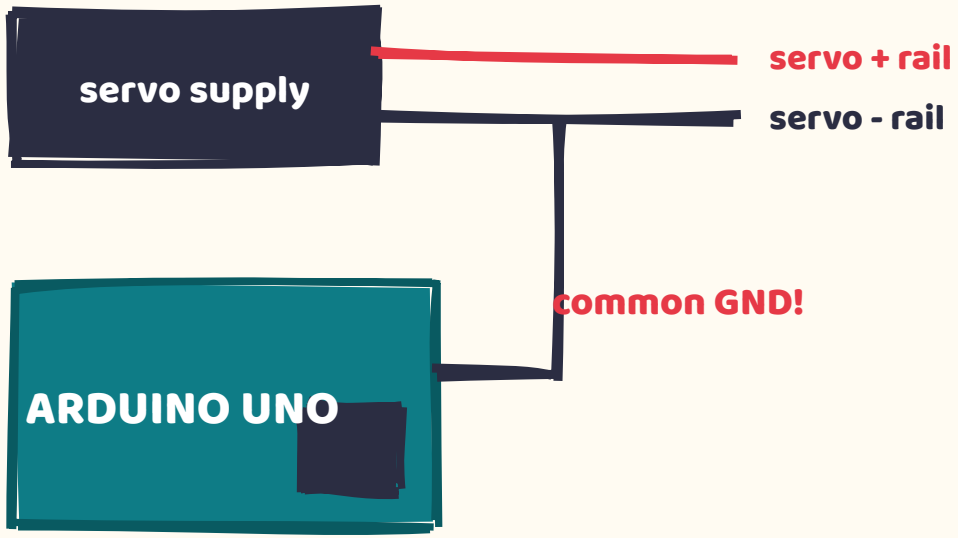
Each EEPROM cell survives about 100,000 writes. Use EEPROM.update(), which only writes if the value changed.

Power

Servos need their own power supply

Why not the Uno's 5V pin?

A stalled MG995 can draw well over **1 A**. Four servos moving together can pull more than USB can give, so the Uno **resets** or behaves strangely.



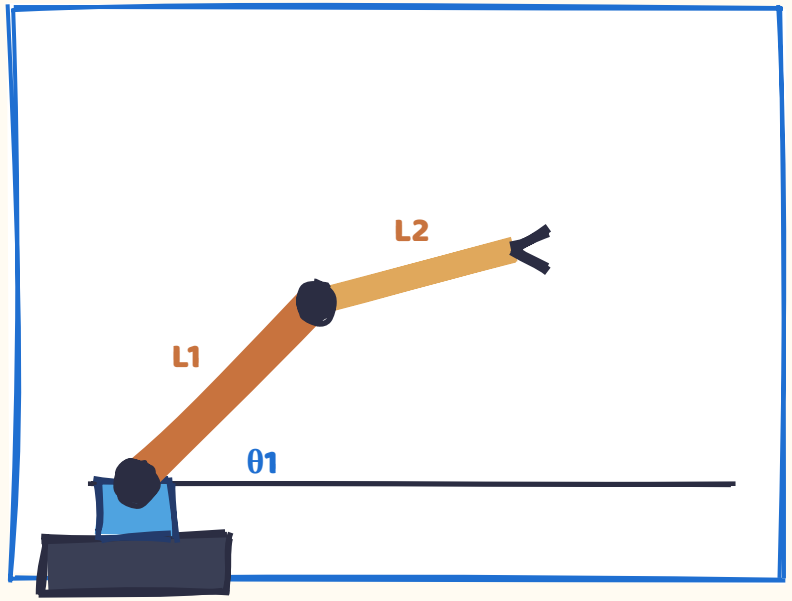
Supply	Voltage	Note
4 × AA rechargeable (NiMH)	about 4.8-5.2 V	best simple option
4 × AA alkaline	about 6 V fresh	OK for MG995, upper limit for SG90
5 V 3 A adapter	5 V	great for the bench
7-12 V adapter + LM2596	set to 5.5 V	the LM2596 needs about 2 V headroom

Rule

Servo red wires → supply +. Servo brown wires → supply − AND Uno GND. Only the orange signal wires go to the Uno.

Maths

How far can the arm reach?



$$x = L1 \cdot \cos \theta1 + L2 \cdot \cos(\theta1 + \theta2)$$
$$y = L1 \cdot \sin \theta1 + L2 \cdot \sin(\theta1 + \theta2)$$

Worked example

L1 = 10 cm, L2 = 8 cm, θ1 = 45°, θ2 = -30°

x = 7.07 + 8·cos 15° = 7.07 + 7.73 = **14.8 cm**

y = 7.07 + 8·sin 15° = 7.07 + 2.07 = **9.1 cm** above the shoulder

Your turn

Measure YOUR L1 and L2. What is the maximum reach? Mark it on the table with tape and test it.

Going further

Working backwards (from x, y to the angles) is called INVERSE kinematics. Phase 2 does it!

Part 2 · Make it

Now let's put it to work!

You know the ideas. Time to play, build and test. Use these slides while you work:

1. Kit per group
2. Centre every servo first
3. Build the base and shoulder
4. Add the forearm, wrist and gripper
5. Wire power and signals
6. Build the puppet controller
7. Live puppet control

Build it

Kit per group

2 × MG995 + 2 × SG90
with horns and screws

4 × 10k potentiometer
the puppet controller

Arduino Uno
+ USB cable

Servo supply
4×AA NiMH or 5 V 3 A

Arm structure
foam board, card or 3D-printed links

2 buttons + 2 × 10k
REC and PLAY

Also: breadboard, M-M and M-F jumpers, glue gun (teacher), ice-cream sticks for the claw, small foam blocks to pick up, ruler and protractor, laptop with Arduino IDE 2.

3D print option
Print the links and gripper in advance (about 2–3 h total) for a stronger arm.

MAKE

Build step 1 of 5

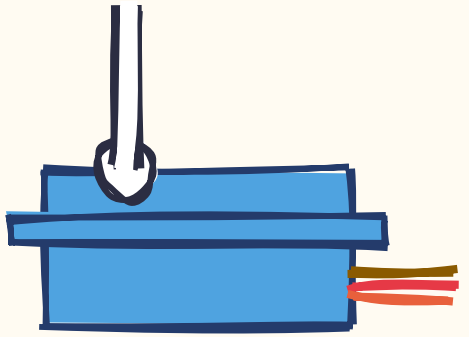
Centre every servo first

a Upload a tiny sketch that sets **all four servos to 90°**. Connect each servo to the supply and its signal pin.

b Only now press the horns on: base horn pointing forward, shoulder horn straight up, wrist straight, gripper half open.

c Write 'centred at 90' on each servo with a marker.

d Turn each servo gently by hand? **No!** Forcing the gears strips them, especially SG90s.



90° = middle

Check: each joint can move about 90° in BOTH directions from its centred position.

Skip this step and your arm will hit its own limits halfway through a movement.

Build step 2 of 5

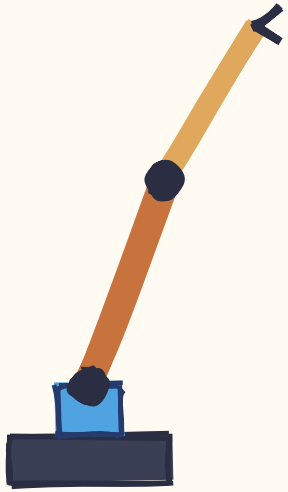
Build the base and shoulder

a Fix the base MG995 into a heavy box or board, shaft up. The base must not tip over: add weight (a small book or stones).

b Glue or screw a round turntable plate to the base horn.

c Mount the shoulder MG995 on the turntable, shaft sideways. Attach the upper arm (about **10 cm**) to its horn.

d Measure and record the upper arm length L1 from shaft centre to elbow.



Check: the shoulder can lift the bare upper arm smoothly from 30° to 150°.

Keep fingers away from the shoulder joint while powered: MG995s are strong.

MAKE

Build step 3 of 5

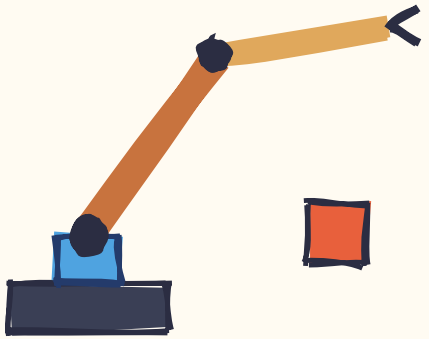
Add the forearm, wrist and gripper

a Attach the forearm (about **8 cm**, L2) with the wrist SG90 at its end.

b Build the gripper: one fixed ice-cream stick and one stick on the gripper SG90 horn.

c Find the gripper's closed and open angles (for example 20° and 110°). Never drive it past closed: a stalled servo heats up.

d Add foam to the gripper tips so it holds blocks without crushing.



Check: the gripper opens and closes on a foam block without buzzing when closed.

A buzzing servo is stalled. Back the angle off by 5-10° immediately.

MAKE

Build step 4 of 5

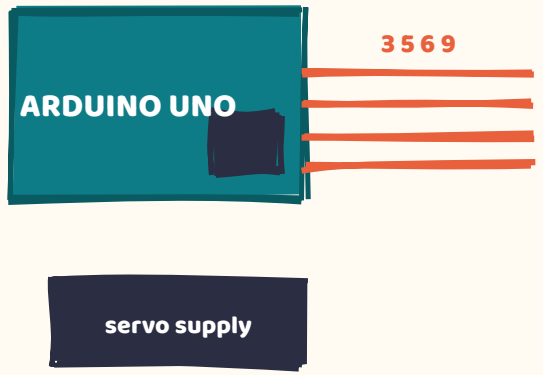
Wire power and signals

a Make a power rail on the breadboard from the servo supply. All four servo red wires to +, brown wires to -.

b Join supply - to the Uno **GND**. Signals: base → 3, shoulder → 5, wrist → 6, gripper → 9.

c Power the Uno from USB. Switch on the servo supply **last**.

d Run the centring sketch again: nothing should jump or reset.



Check: the Uno never resets when all four servos move together.

If the Uno resets when servos move, the servos are still powered from the Uno.

Build step 5 of 5

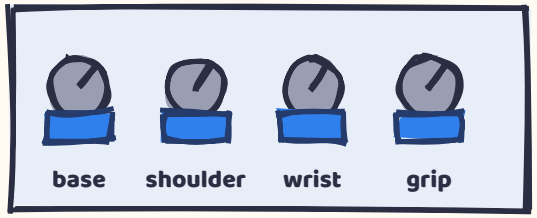
Build the puppet controller

a Mount 4 potentiometers on a card panel, laid out like the arm's joints (base at the bottom, grip at the top).

b Each pot: outer legs to **5V** and **GND**, middle leg to **A0** (base), A1 (shoulder), A2 (wrist), A3 (grip).

c Add two buttons with 10k pull-down resistors: **REC → pin 2**, **PLAY → pin 4**.

d Print the four pot values to the Serial Monitor: each should sweep from about 0 to 1023.





Check: turning each knob changes only its own number.

Pots use the Uno 5V (tiny current) – that is fine. Only servos need the separate supply.

Code it: part 1

Live puppet control

```
#include <Servo.h>
#include <EEPROM.h>

Servo joint[4];
const byte SERVO_PIN[4] = {3, 5, 6, 9};
const byte POT_PIN[4]   = {A0, A1, A2, A3};
const byte MIN_A[4] = {0, 30, 0, 20};      // safe limits
const byte MAX_A[4] = {180, 150, 180, 110};  // (gripper!)
const byte REC = 2, PLAY = 4;
byte angle[4];

void setup() {
  for (int i = 0; i < 4; i++) joint[i].attach(SERVO_PIN[i]);
  pinMode(REC, INPUT); pinMode(PLAY, INPUT);
  pinMode(LED_BUILTIN, OUTPUT);
  Serial.begin(9600);
}

void readKnobs() {
  for (int i = 0; i < 4; i++) {
    angle[i] = map(analogRead(POT_PIN[i]), 0, 1023, MIN_A[i], MAX_A[i]);
    joint[i].write(angle[i]);
  }
}
```

Arrays

One array per property (pin, limits, angle) and a loop for all 4 joints. Adding a 5th joint = one more number.

Safe limits

map() into each joint's **own** range. The gripper never squeezes past 20°, the shoulder never hits the table.

Test

Call readKnobs() in loop() with delay(15) and puppet the arm around.

Code it: part 2

Record poses to EEPROM

```
byte count = 0;                          // poses saved

void recordPose() {
  if (count >= 50) return;               // memory full
  for (int i = 0; i < 4; i++)
    EEPROM.update(1 + count * 4 + i, angle[i]);
  count++;
  EEPROM.update(0, count);
  digitalWrite(LED_BUILTIN, HIGH); delay(300);   // flash = saved
  digitalWrite(LED_BUILTIN, LOW);
}

void loop() {
  readKnobs();
  if (digitalRead(REC) == HIGH && digitalRead(PLAY) == HIGH) {
    delay(2000);                         // hold both = erase
    count = 0; EEPROM.update(0, 0);
  } else if (digitalRead(REC) == HIGH) {
    recordPose(); delay(400);            // one save per press
  } else if (digitalRead(PLAY) == HIGH) {
    replay();
  }
  delay(15);
}
// in setup(): count = EEPROM.read(0); if (count > 50) count = 0;
```

Address maths

Pose n, joint i lives at address **1 + n×4 + i**. Address 0 holds the count.

First-run safety

A brand-new Uno's EEPROM contains 255s. The check in setup() treats that as 'no recording'.

Feedback

The LED flash tells the operator 'saved'. Good machines always confirm actions.

Code it: part 3

Smooth replay (interpolation)

```
void moveSmooth(byte target[4]) {
  byte start[4];
  for (int i = 0; i < 4; i++) start[i] = joint[i].read();
  for (int step = 1; step <= 25; step++) {      // 25 small steps
    for (int i = 0; i < 4; i++)
      joint[i].write(start[i] + (target[i] - start[i]) * step / 25);
    delay(20);                                  // 25 × 20 ms = 0.5 s
  }
}

void replay() {
  byte pose[4];
  for (int n = 0; n < count; n++) {
    for (int i = 0; i < 4; i++) pose[i] = EEPROM.read(1 + n * 4 + i);
    moveSmooth(pose);
    delay(300);                                 // pause at each pose
  }
}
```

Interpolation

Instead of jumping, the arm takes 25 tiny steps between poses. Smoother, safer and kinder to the gears.

Speed control

Change 25 or the 20 ms delay to move faster or slower.

Order matters

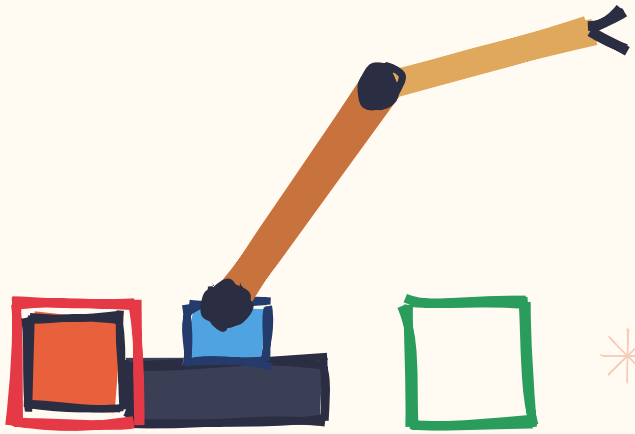
Record an extra 'lift' pose before moving sideways, or the block drags across the table!

MAKE

The challenge

Pick-and-place task

1. Tape a START square and a FINISH square 15 cm apart.
2. Record: home → above block → open → down → close → up → over finish → down → open → up → home.
3. Press PLAY and let it move the block. Hands off!
4. Repeat 10 times without re-recording. Measure where the block lands each time.



About 10 poses

A good pick-and-place uses roughly 8–12 poses.

Measure performance

Accuracy vs repeatability

Run	X error	Y error	Block dropped?
Run 1	____ mm	____ mm	yes / no
Run 2	____ mm	____ mm	yes / no
Run 3	____ mm	____ mm	yes / no
Run 4	____ mm	____ mm	yes / no
Run 5	____ mm	____ mm	yes / no
Run 6	____ mm	____ mm	yes / no

Accuracy

How close the block lands to the **target** centre (average error).

Repeatability

How close the landings are to **each other**. Factory arms repeat to about 0.05 mm!

Improve it

Slower moves, a lighter forearm, firmer joints and a 'settle' pause before opening.

Test like an engineer

Troubleshooting

Symptom	Likely cause / fix
Arm twitches / jitters	noisy pot readings: average 4 readings, or add 100 nF capacitor
Uno resets	servos powered from the Uno: use the separate supply
Servo buzzes and gets hot	stalled at a limit: shrink MIN/MAX for that joint
Replay does nothing	count is 0: record first, or check EEPROM check in setup()
Arm sags under load	joint too weak or link too long: shorten, lighten, or use MG995

Debug checklist

Print the 4 angles to Serial while puppeting.
Test each joint alone before all four together.

Keep a spare SG90 ready. Stripped gears are normal in arm projects; that is why we set limits.

MAKE

Level up!

Level up

Choose your challenge

Loop mode: replay forever until a button is pressed, like a real production line.

Speed per pose: store a speed byte with each pose (5 bytes per pose).

Sense the block: add an IR sensor so the arm waits until a block arrives.

PC teach pendant: send poses from a Python or Processing slider window over serial.

Think like an engineer

A factory arm works next to people. What safety features would you add before a human could stand nearby?

Boss level

Real cobots stop when they feel unexpected force. Try: stop if a servo's angle doesn't match its command.

Show what you know

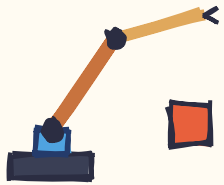
What did we learn?

- [x] Each joint is one degree of freedom
- [x] Torque = force × distance: strong servos near the base
- [x] map() turns knob values into safe joint angles
- [x] EEPROM keeps poses after power-off; use update()
- [x] Interpolation makes motion smooth and safe

Exit ticket: why does the shoulder need a stronger servo than the gripper?

At home

Watch a video of a factory robot arm. Count its joints (degrees of freedom).



Next class: Fall Detection Wearable for the Elderly!